

VELOX

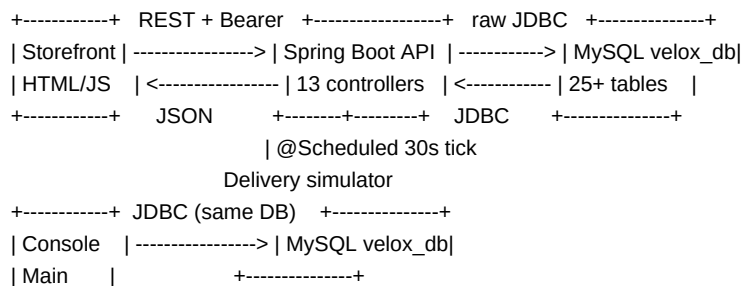
Technical Architecture and API Reference

All-in-One Express Delivery: Spring Boot REST API, console application, web storefront and MySQL database sharing one database as the single source of truth.

Version 1.0 - September 2026 - Covers KAN-60 (QA, Testing & Documentation), KAN-90 (order history & tracking), KAN-126 (unit + E2E tests), KAN-130 (this document).

Stack: Java 17, Spring Boot 3.3, raw JDBC, MySQL 8, static HTML/CSS/JS frontend.

1. System architecture



Layer	Technology	Notes
API	Spring Boot 3.3, Java 17, Tomcat :8080	CORS open, Bearer tokens, BCrypt only (no Security auto-config)
Persistence	Raw JDBC via DatabaseConnection	No JPA; queries in dao/*, manual ResultSet mapping
Auth	Email + password + 6-digit OTP	Inactive until OTP verified; tokens in sessions table
Background	DeliverySimulationService, 30s	PENDING (>60s) > PROCESSING > IN_TRANSIT > ARRIVED > DELIVERED
Frontend	Static HTML/CSS/JS, AR/EN + RTL/LTR	API_BASE_URL=http://localhost:8080/api

Order tracking state machine (forward-only)

PENDING -> PROCESSING -> SHIPPED -> DELIVERED. SHIPPED is stored and shown as IN_TRANSIT; ARRIVED is an intermediate simulation step. Skipped steps are rejected with 400.

Registration and login flow

POST /auth/register creates an inactive account and returns {pending:true, otp} (dev). POST /auth/verify-otp activates it and returns {token, user}. POST /auth/login rejects unverified accounts with 401.

2. API reference (prefix /api, JSON, errors as {message})

Auth

Method + path	Auth	Contract
POST /auth/register	no	{email,password,full_name,phone_number,governorate} > {pending,otp}
POST /auth/verify-otp	no	{email,code} > {token,user}
POST /auth/resend-otp	no	{email} > {email,otp}

Method + path	Auth	Contract
POST /auth/login	no	{email,password} > {token,user}; 401 if unverified
POST /auth/logout	Bearer	revokes token
GET /auth/profile?userId=	self	profile + loyalty + offers
PUT /auth/profile	self	update name/phone/governorate

Catalog and governorates

Method + path	Auth	Contract
GET /products	no	catalog with images + Arabic names
GET /categories	no	food/fashion/electronics + counts
GET /governorates	no	27 governorates, codes 1-27, prices
GET /governorates/{name}	no	single governorate (case-insensitive)
GET /governorates/{name}/shipping	no	{governorate,shippingPrice,deliveryDays}

Orders (KAN-90)

Method + path	Auth	Contract
GET /orders/history?userId=&page=&size=	self	paged history; 403 for other users
POST /orders	Bearer	{items:[{product_id,quantity}],governorate?,paymentMethod?}
GET /orders/{id}/tracking	owner	timeline + driver location; 401/403/404
PATCH /orders/{id}/status?status=	owner	forward-only; 400 on illegal jump
GET /orders/{id}/invoice	owner	application/pdf, Arabic-capable

Wallet, loyalty, feedback

Method + path	Auth	Contract
GET /wallet/balance?userId=	self	remaining_budget
POST /wallet/topup	Bearer	{amount}
GET /loyalty/status?userId=	self	progress to free delivery
GET /offers/personalized?userId=	self	offers from history
POST /reviews POST /complaints	Bearer	{orderId?, rating details}

3. Database schema map (velox_db)

Table	Key columns
users	id, full_name, email, password_hash, phone_number, governorate, is_verified, is_active, budgets, role
orders	id, order_code, user_id, status, totals, delivery_fee, final_amount, shipping_address, order_date
order_items	id, order_id, product_id, quantity, unit_price, subtotal
products	id, store_id, category_id, name(+_ar), description(+_ar), image_url, price, stock, type, size
sessions	token, email, expires_at
otp_codes	id, email, code_hash, expires_at, attempts, used
deliveries	id, order_id, driver_id, zone, delivery_status, delivery_fee
driver_locations	driver_id, latitude, longitude, updated_at
invoices	invoice_id, order_id, user_id, invoice_pdf_path
return_requests	id, order_id, user_id, status

Table	Key columns
user_notifications	id, user_id, title, message, type, is_read
user_payment_cards	id, user_id, method_type, last_4_digits, brand, provider

Seed: VELOX_D2.SQL base dump plus migrations V2 (governorates, SHIPPED, sessions, images), V3 (deliveries, nullable links), V4 (returns, cards), V5 (notifications), V6 (payment methods, OTP). Reference catalog holds about 25 products.

4. Installation (any machine)

Requirements: JDK 17+, MySQL 8 running, a browser, internet on first Maven run.

1) Create empty database velox_db. 2) Import velox-db/VELOX_D2.SQL, then migrations/ V2..V6 in numeric order. 3) Copy application.properties.example to application.properties (git-ignored) and set the MySQL password, or export VELOX_DB_URL/USER/PASSWORD. 4) cd velox-backend and run: mvn spring-boot:run. Wait for Started VeloxApplication and open <http://localhost:8080/api/governorates/CAIRO>. 5) Double-click velox-frontend/index.html, register, confirm the OTP, order. Console app: mvn exec:java. On MariaDB, swap utf8mb4_0900_ai_ci to utf8mb4_unicode_ci for the local import only.

5. QA summary (KAN-126)

Unit suite: 35 tests, all passing (mvn test). JUnit 5 + Mockito; the legacy JUnit 4 ReturnServiceTest runs on the vintage engine.

Test class	Tests	Coverage
enums.GovernorateTest	9	27 governorates, codes, prices, lookups, rejections
controller.GovernorateControllerTest	5	MockMvc list/lookup/404/shipping
controller.OrderControllerTest	9	Mockito service: 200/400/401/403/404 paths
service.OrderServiceTest	8	Mocked JDBC: paging, mapping, transition guard
dto.OrderDtoTest	3	DTO mapping, paging envelope, timeline
service.ReturnServiceTest	1	legacy return flow

E2E: velox-backend/postman/VELOX_API_Tests.postman_collection.json. Import into Postman and run Auth, Governorates, Orders, Catalog in order; every request carries success and failure assertions (401/403/404/400). All scenarios were executed against a live server.